Product Requirements Document (PRD)

ClaimX — Multi-Line Insurance Claims Automation Platform

Document Type	Product Requirements Document (PRD)
Product Name	ClaimX
Version	1.0 (Draft)
Team	Preetham S Gowda, Naushin, Chinmaya Bharadwaj S V, Chandana M P
Status	Draft for review

Tech Stack

Layer	Technology
Backend Framework	Django + Django REST Framework
AI Orchestration	LangGraph (deterministic graph mode) / CrewAI (agentic mode)

LLM / Extraction	Gemini 2.5 Flash, with regex-based deterministic fallback
OCR	Tesseract OCR + pdf2image / Poppler
Database	MongoDB
Report Generation	ReportLab (PDF)
Frontend	Vanilla HTML / CSS / JavaScript
Authentication	JWT (`Authorization: Bearer <token>`)
Containerization	Docker + Docker Compose
Deployment	Cloud Blueprint-based deployment (e.g. Render)
CI/CD	GitHub Actions (install, checks, smoke tests, deploy-on-merge)

---

1. Purpose & Background

ClaimX is an AI-driven insurance claims automation platform designed to handle the **entire claim lifecycle** — from document upload through fraud detection, verification, payout estimation, and settlement report generation — across multiple insurance products, not just one. Insurance claimants today deal with different portals, paperwork formats, and manual verification processes depending on the product they're claiming against. ClaimX consolidates this into **one platform, one AI pipeline, one set of dashboards** that adapts its behavior based on the insurance type being claimed.

2. Problem Statement

Manual claim processing is slow, inconsistent across insurance products, and heavily paperwork-dependent. Fraud screening varies in rigor depending on who reviews a claim. Claimants have no single place

to track claims spanning different policies they hold. ClaimX solves this by automating document intake, extraction, and fraud/verification checks with AI, while keeping payout logic transparent, rule-based, and auditable per insurance type.

3. Goals & Objectives

- Support claim intake and automated processing for **7 insurance types** (below) through one unified platform.
- Run every claim through a consistent pipeline — OCR → Classify → Extract → Fraud-check → Verify → Estimate → Report → Store — made **insurance-type aware** at each stage.
- Support three user roles: **Claimant, Agent, Admin**.
- Keep the payout/estimation logic transparent, rule-based, and auditable per insurance type (no black-box payout decisions).
- Provide strong security and privacy defaults: authenticated access, OTP-verified registration, and masking of sensitive identifiers at rest.

4. Scope

In scope

- Claim submission and automated processing for: Life, Health, Motor, Home, Travel, Personal Accident, and Liability/Business insurance.
- Insurance-type-aware document classification and field extraction.
- Type-specific fraud signals and payout/estimation rules.
- Agent review queue and Admin approval workflow across all types.
- Unified claimant dashboard showing claims across all insurance types.

Out of scope (v1)

- Real-time payment disbursement / bank settlement integration.

- Insurer-side policy issuance or premium collection (system only consumes existing policy records).
- Legal adjudication for liability claims (system estimates exposure only; final settlement is routed to a human agent/legal reviewer).
- Mobile native apps (web-only for v1).

5. User Roles & Personas

Role	Description
Claimant	Registers, verifies email via OTP, uploads documents, selects insurance type, tracks claim status, downloads settlement report
Agent	Reviews AI-processed claims queued by type, validates flagged fraud cases, submits recommendation
Admin	Manages agents, approves/rejects claims, views platform-wide analytics across all insurance lines

6. Supported Insurance Lines & Claim Logic

Each insurance type plugs into the same AI pipeline via a type-specific **document set, extraction schema, fraud rule-set, and estimation strategy**, so the whole platform stays a single codebase while behaving correctly per product.

#	Insurance Type	Claim Sub-Types	Key Documents Required	Payout / Estimation Logic

1	**Life Insurance**	Natural death, Accidental death, ULIP maturity	Death certificate, policy document, Aadhaar, PAN, bank proof, nominee ID, FIR (if accidental), hospital record	Natural = 100% Sum Assured; Accidental = 200% (double indemnity)
2	**Health Insurance**	Cashless, Reimbursement, Critical illness	Discharge summary, medical bills, prescriptions, diagnostic reports, policy document, pre-authorization form	Sum of eligible bills up to Sum Insured, minus co-pay/ deductible; critical illness = lump-sum % of SI by illness category
3	**Motor Insurance**	Own damage, Total loss, Third-party	RC, driving license, FIR/ accident report, garage repair estimate, damage photos, policy document	Own damage = repair estimate – depreciation – deductible, capped at IDV; Total loss = IDV; Third-party = flagged for agent/ tribunal review (no fixed formula)

4	**Home Insurance**	Structure damage, Content loss/theft	Ownership proof, FIR (theft), damage photos/video, repair/ replacement estimate, municipal/fire report (disaster)	Lower of (repair/ replacement cost, Sum Insured) minus policy excess
5	**Travel Insurance**	Medical emergency, Trip cancellation/ delay, Baggage loss	Itinerary/ boarding pass, passport, medical bills, airline delay/ cancellation certificate, baggage loss report (PIR)	Medical = actual bills up to sub-limit; Cancellation = fixed policy-schedule slabs; Baggage = per-item schedule up to SI
6	**Personal Accident Insurance**	Accidental death, PTD, PPD, TTD	FIR, death certificate or disability certificate, hospital records, policy document	Death/PTD = 100% Capital Sum Insured; PPD = % of CSI per disability schedule table; TTD = weekly benefit × weeks, capped

7	**Liability / Business Insurance**	Professional indemnity, Public liability, Errors & omissions	Legal notice/ demand letter, contract/ engagement agreement, incident report, business registration proof, policy document	System computes exposure ceiling = min(claimed amount, policy limit) – deductible; always routed to Agent/legal review — no auto-settlement

7. Functional Requirements

7.1 Authentication & Onboarding

- Claimant self-registration with email OTP verification.
- Token-based login for Claimant / Agent / Admin.
- Agent accounts created only by Admin; Admin account seeded during initial setup.

7.2 Claim Intake

- Claimant selects **insurance type** and **claim sub-type** before document upload.
- Multipart upload of PDF/PNG/JPG/JPEG documents, validated against the required-document checklist for the selected type.

7.3 AI Processing Pipeline

Stage	Behavior

Input Validator	Checks file presence/type, and validates against the selected insurance type's required-document list
OCR Processor	Extracts text from scanned images/PDFs
Document Classifier	Detects document type using a keyword/layout dictionary specific to the selected insurance type
Data Extractor	LLM-assisted structured extraction (with a deterministic fallback), using a **type-specific field schema** (e.g., IDV & RC number for motor, illness category for health)
Identity Normalizer	Normalizes Aadhaar/PAN/date formats, reused across all types
Fraud Detector	Base rule set (identity/date/format checks) + **type-specific signals** (see §8)
Policy Verifier	Cross-checks extracted identity against stored policy records, filtered by `insurance_type` + `policy_sub_type`
Claim Estimator	**Strategy-pattern estimator** — one estimation module per insurance type (§6), selected by `insurance_type`
Report Generator	Auto-generates a PDF settlement report, template variant per insurance type
Mask + Storage	Sensitive identifiers (Aadhaar/PAN/account) masked before persistence

7.4 Agent Review

- Agent dashboard queue filterable by insurance type and fraud-flag status.
- Agent can view AI-extracted data, override estimated payout with justification, and submit a recommendation.
- Liability/Business claims and Motor third-party claims always route to Agent review (never auto-approved).

7.5 Admin Panel

- Platform-wide claims table filterable by insurance type, status, and fraud flag.
- Approve/reject claims, manage agent accounts, view per-type analytics (claim volume, average payout, fraud rate).

7.6 Reporting

- Auto-generated PDF settlement report per claim, template selected by insurance type, downloadable by Claimant/Agent/Admin.

8. Fraud Detection — Base + Type-Specific Signals

Base signals: name mismatch across documents, invalid Aadhaar/PAN format, low OCR confidence, date inconsistencies, suspicious/altered text formatting.

Type-specific additions:

- **Health:** inflated bill amounts, duplicate bill submission, hospital–diagnosis mismatch.
- **Motor:** RC–policy vehicle mismatch, inflated repair estimate, pre-existing damage visible in photos, expired driving license.
- **Home:** repeated claims for the same peril, estimate inflation vs. property value on record.
- **Travel:** travel dates falling outside policy validity window, duplicate baggage-loss claims.
- **Personal Accident:** FIR–hospital timeline mismatch, inconsistent disability certificate details.
- **Liability:** claimed amount grossly exceeding policy limit, inconsistent dates across legal correspondence.

9. Non-Functional Requirements

Category	Requirement
Security	Token-based auth; Aadhaar/PAN/bank account masked before persistence
Performance	Low OCR confidence triggers a `failed_ocr` status with a re-upload prompt rather than a bad extraction
Reliability	Deterministic extraction fallback if the LLM API is unavailable or a call fails
Portability	Containerized local dev environment; cloud-deployable via a standard blueprint/manifest
Auditability	Every claim decision (approve/reject/estimate) stored with basis text, no silent overrides
CI/CD	Automated checks on push (install, lint/checks, smoke test) and deploy-on-merge

10. System Architecture & Tech Stack

- **Backend:** Django + Django REST Framework
- **AI Orchestration:** Graph-based deterministic orchestration (LangGraph) and multi-agent role-based orchestration (CrewAI), selectable per deployment
- **LLM:** Gemini 2.5 Flash for structured extraction, with a regex-based fallback
- **OCR:** Tesseract OCR + pdf2image/Poppler
- **Database:** MongoDB, with `insurance_type` woven through the relevant collections
- **Report Generation:** ReportLab (PDF)
- **Frontend:** Vanilla HTML/CSS/JS, served by Django
- **Auth:** JWT (`Authorization: Bearer <token>`)
- **Deployment:** Docker + Docker Compose locally; cloud Blueprint deployment in production; CI/CD via GitHub Actions

11. Data Model (MongoDB)

Collection	Purpose
`users`	Claimant accounts
`otp_verification`	OTP codes for email verification
`policy_holder_data`	Pre-existing insurance details, incl. `insurance_type`, `policy_sub_type`, and type-specific fields (e.g. `idv`, `sum_insured`, `capital_sum_insured`)
`nominee_details`	Nominee information (Life & Personal Accident)
`claims`	Processed claims with AI results, incl. `insurance_type`, `claim_sub_type`, `type_specific_extracted_data`
`claim_documents`	File paths for uploaded documents, incl. `insurance_type` for filtering
`agents`	Agent accounts
`admins`	Admin accounts
`fraud_logs`	Fraud detection audit trail, incl. `insurance_type`, `signal_category` (base vs. type-specific)
`estimation_rules`	Config-driven payout tables per type (e.g., PPD disability schedule, baggage-loss schedule, cancellation slabs) — keeps estimation logic auditable and adjustable without code changes

12. API Design

Endpoint	Method	Description

`/api/auth/register/`, `/login/`, `/verify-otp/`, `/resend-otp/`, `/profile/`	POST/ GET	Registration, OTP verification, login, profile
`/api/pipeline/process/`	POST	Upload files & run the full AI pipeline; requires `insurance_type` in the multipart payload so the matching document dictionary, extraction schema, and estimator get loaded
`/api/claims/list/`	GET	List own claims, filterable by `?insurance_type=`
`/api/claims/<id>/`, `/status/`	GET/ PATCH	Claim detail and status update
`/api/reports/<id>/download/`	GET	Download settlement PDF
`/api/agent/claims/`, `/claims/<id>/review/`	GET/ POST	Agent claim queue (filterable by insurance type) and review submission
`/api/admin-panel/claims/`	GET	All claims, filterable by `?insurance_type=` and `?claim_sub_type=`
`/api/admin-panel/claims/<id>/approve/`, `/reject/`	PATCH	Admin approval workflow
`/api/admin-panel/agents/create/`	POST	Create agent account

13. Frontend Pages

Page	Role	Notes
Landing / Register / Login / Verify-OTP	Public	Standard onboarding flow
Claimant Dashboard	Claimant	Insurance-type selector at "New Claim" step; claim list shows a type badge
New Claim	Claimant	Dynamic document checklist generated from the selected insurance type
Agent Dashboard	Agent	Insurance-type filter tabs on the claims queue
Admin Dashboard	Admin	Per-type analytics widgets alongside the global claims table

14. Success Metrics

- Successful automated extraction rate ≥ 85% across all 7 insurance types (excluding `failed_ocr` cases).
- Fraud detection recall validated against a labeled test set per type.
- Median time from upload to settlement-report generation, tracked per insurance type.
- Agent override rate on AI-estimated payout (lower is better, indicates trust in automation).

15. Suggested Team Split *(draft — to be finalized by the team)*

Area	Suggested Owner(s)
Backend (Django apps, APIs, MongoDB schema)	Preetham S Gowda

AI Pipeline (orchestration, classifier, extractor, fraud rules, estimators)	Chinmaya Bharadwaj S V
Frontend (dashboards, new-claim flow, agent/ admin UI)	Chandana M P
QA, Documentation, Report Templates & Deployment (Docker/CI)	Naushin

16. Risks & Assumptions

- Assumes access to an LLM API with sufficient quota for 7-type extraction volume; the regex fallback mitigates outages but with lower accuracy.
- Motor third-party and Liability claims have no deterministic payout formula — the system deliberately avoids auto-settlement for these and routes to human review, which must be communicated clearly in the UI to avoid user confusion.
- Type-specific fraud rules (§8) are heuristic, not exhaustive, and should be tuned against real/sample claim data before being treated as authoritative.
- Assumes seed policy-holder data is available across all 7 types before pipeline testing.

17. Milestones (suggested)

Phase	Deliverable
1	Data model (`insurance_type` fields) + insurance-type selector on claim intake
2	Document Classifier + Data Extractor with per-type schemas (start with Life + Health + Motor)
3	Type-specific Claim Estimators + `estimation_rules` collection

4	Fraud Detector extended with type-specific signals
5	Remaining insurance types (Home, Travel, Personal Accident, Liability)
6	Agent/Admin dashboard filters + analytics; report template variants
7	End-to-end smoke tests per type; CI/CD; deployment

ClaimX — a unified multi-line insurance claims automation platform covering Life, Health, Motor, Home, Travel, Personal Accident, and Liability/Business insurance.